PostgreSQL
COMPLETE NOTES

Database Fundamentals → Advanced SQL → Optimization

Parts 1 – 5 | From Basics to Interview-Ready

Premium Structured Study Guide
Colored Notes • Diagrams • Real-World Examples

Includes: Data Types • Constraints • CRUD • Filtering • Aggregation
JOINs • Subqueries • Window Functions • CTEs • Indexes • Transactions
Functions • Triggers • JSONB • Arrays • Query Optimization

Table of Contents

Part 1 — Fundamentals, Data Types, Constraints & Basic SQL
Part 2 — Data Retrieval, Filtering, Sorting & Aggregate Functions
Part 3 — JOINs, Relationships & Subqueries
Part 4 — Window Functions, CTEs, Views, Indexes & Transactions
Part 5 — Functions, Procedures, Triggers, JSONB, Arrays & Optimization

1 PART 1 — Fundamentals, Data Types, Constraints & Basic SQL

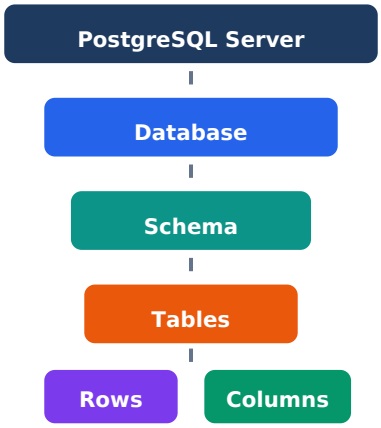
1. PostgreSQL Basics

PostgreSQL is an open-source relational / object-relational database management system. SQL is the language used to communicate with the database.

Core Terminology

Term	Meaning
PostgreSQL	Open-source RDBMS
SQL	Language to talk to the database
Database	Collection of related data
Table	Stores data in rows & columns
Row	One complete record
Column	One attribute / field
Schema	Namespace to organize objects inside a DB
Primary Key	Uniquely identifies each row
Foreign Key	Creates relationship between two tables

Hierarchy



PostgreSQL Server → Database → Schema → Tables → Rows + Columns

2. Numeric Data Types

Data Type	Storage	Use / Example
SMALLINT	2 bytes	Small integers (−32 768 … 32 767)
INTEGER / INT	4 bytes	Normal integers (−2.1B … 2.1B)
BIGINT	8 bytes	Very large integers
NUMERIC(p,s)	Variable	Exact decimal — prefer for money
DECIMAL(p,s)	Variable	Same as NUMERIC
REAL	4 bytes	Approximate floating point
DOUBLE PRECISION	8 bytes	Higher-precision approximate

```sql
NUMERIC(10,2)   -- 10 total digits, 2 after decimal
-- Example valid value: 12345678.90
```

3. Character / String Types

Type	Meaning	Typical Use
CHAR(n)	Fixed-length string	Country codes, fixed codes
VARCHAR(n)	Variable length with limit	Names, emails
TEXT	Unlimited variable text	Descriptions, long content

```sql
name VARCHAR(100)
email VARCHAR(150)
description TEXT
country_code CHAR(2)
```

4. Boolean, Date/Time, UUID, ENUM, JSONB, Arrays

Boolean

BOOLEAN → TRUE / FALSE / NULL. PostgreSQL also accepts t / f.

Date & Time

Type	Stores	Example
DATE	Date only	2026-09-21
TIME	Time only	14:30:00
TIMESTAMP	Date + time	2026-09-21 14:30:00
TIMESTAMPTZ	Timestamp with time zone	2026-09-21 14:30:00+05:30
INTERVAL	Duration	2 days, 3 hours

UUID

Universally Unique Identifier — sequential integer ki jagah unique non-guessable IDs ke liye. Example: `550e8400-e29b-41d4-a716-446655440000`

ENUM

```sql
CREATE TYPE order_status AS ENUM (
    'pending', 'shipped', 'delivered', 'cancelled'
);
-- column: status order_status
```

JSON / JSONB & Array

JSONB binary format — querying & indexing ke liye preferred. Arrays: `skills TEXT[]` → {'Python','SQL','ML'}

5. Most Important Data Types (Quick List)

INTEGER • BIGINT • NUMERIC • VARCHAR • TEXT • BOOLEAN • DATE • TIMESTAMP / TIMESTAMPTZ • UUID • JSONB • ARRAY

6. Constraints

Constraint	Purpose
NOT NULL	Value compulsory
UNIQUE	Duplicate values prevent (NULLs usually allowed multiple)
PRIMARY KEY	Row uniquely identify (unique + not null)
FOREIGN KEY	Tables ke beech relationship
CHECK	Custom condition enforce
DEFAULT	Value automatically provide

```SQL
CREATE TABLE students (
    student_id INTEGER PRIMARY KEY,
    name       VARCHAR(100) NOT NULL,
    email      VARCHAR(150) UNIQUE,
    age        INTEGER CHECK (age >= 18),
    city       VARCHAR(50) DEFAULT 'Delhi'
);
```

Auto-Generated ID (Modern)

```SQL
CREATE TABLE customers (
    customer_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name        VARCHAR(100) NOT NULL
);
```

INSERT ke time ID manually dene ki zarurat nahi — PostgreSQL khud generate karta hai.

7. Basic CRUD Commands

CREATE / DROP Database & Table

```SQL
CREATE DATABASE ecommerce_db;
DROP DATABASE ecommerce_db;   -- ⚠ permanent
```

```SQL
CREATE TABLE products (
    product_id   INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    product_name VARCHAR(100) NOT NULL,
    category     VARCHAR(50) NOT NULL,
    price        NUMERIC(10,2) CHECK (price >= 0),
    stock        INTEGER CHECK (stock >= 0),
    is_active    BOOLEAN DEFAULT TRUE
);
```

INSERT

```SQL
INSERT INTO products (product_name, category, price, stock)
VALUES
    ('Laptop', 'Electronics', 65000, 10),
    ('Mouse',  'Electronics', 800, 50),
    ('Chair',  'Furniture', 5000, 20);
```

SELECT / WHERE / UPDATE / DELETE

```sql
SELECT product_name, price FROM products WHERE price > 5000;

UPDATE products SET price = 60000, stock = 15 WHERE product_id = 1;

DELETE FROM products WHERE product_id = 3;
```

DELETE vs TRUNCATE vs DROP

Command	Removes rows?	Removes table?
DELETE	Selected / all rows	No
TRUNCATE	All rows (fast)	No
DROP TABLE	All data	Yes

ALTER TABLE

```sql
ALTER TABLE products ADD COLUMN brand VARCHAR(100);
ALTER TABLE products RENAME COLUMN brand TO manufacturer;
ALTER TABLE products ALTER COLUMN product_name TYPE VARCHAR(200);
ALTER TABLE products DROP COLUMN manufacturer;
ALTER TABLE products RENAME TO product_details;
```

8. SQL Command Categories

Category	Commands	Purpose
DDL	CREATE, ALTER, DROP, TRUNCATE	Structure
DML	INSERT, UPDATE, DELETE	Data modification
DQL	SELECT	Data retrieval
DCL	GRANT, REVOKE	Permissions
TCL	COMMIT, ROLLBACK, SAVEPOINT	Transactions

9. Part 1 — Practice

ℹ NOTE

customers table banao (customer_id auto + PK, name required, email unique, age ≥ 18, is_active default TRUE, created_at current timestamp). 8–10 records insert karke SELECT, WHERE, UPDATE, DELETE practice karo.

2 PART 2 — Retrieval, Filtering, Sorting & Aggregates

1. SELECT, DISTINCT, Alias

```sql
SELECT name, email, city FROM customers;
SELECT name AS customer_name, email AS customer_email FROM customers;
SELECT DISTINCT city FROM customers;
SELECT DISTINCT city, age FROM customers;   -- combination unique
```

2. WHERE + Comparison Operators

= <> != > < >= <=

```sql
SELECT * FROM products WHERE price >= 50000;
```

3. AND / OR / NOT / Parentheses

```sql
SELECT * FROM employees
WHERE department = 'IT' AND (salary > 50000 OR city = 'Delhi');
```

4. IN, BETWEEN, LIKE, ILIKE, NULL

```sql
WHERE city IN ('Delhi', 'Agra', 'Mumbai');
WHERE city NOT IN ('Delhi', 'Agra');
WHERE price BETWEEN 1000 AND 5000;          -- inclusive
WHERE name LIKE 'A%';                       -- starts with A
WHERE name LIKE '%an%';                     -- contains
WHERE name LIKE '_man';                     -- exactly one char + man
WHERE name ILIKE 'gaurav';                  -- case-insensitive (PG specific)
WHERE phone IS NULL;
WHERE phone IS NOT NULL;
```

5. ORDER BY, LIMIT, OFFSET

```sql
SELECT * FROM products ORDER BY price DESC;
SELECT * FROM employees ORDER BY department ASC, salary DESC;
SELECT * FROM products ORDER BY price DESC LIMIT 3;   -- top 3
SELECT * FROM products LIMIT 10 OFFSET 20;            -- page 3
```

Pagination: Page 1 → LIMIT 10 OFFSET 0 | Page 2 → LIMIT 10 OFFSET 10 | ...

6. Aggregate Functions

Function	Purpose
COUNT(*) / COUNT(col)	Number of rows / non-null values
SUM(col)	Total of numeric values
AVG(col)	Average
MIN(col) / MAX(col)	Minimum / Maximum

Function	Purpose
COUNT(DISTINCT col)	Unique values count

```sql
SELECT COUNT(*) FROM customers;
SELECT COUNT(DISTINCT city) FROM customers;
SELECT AVG(salary) FROM employees WHERE department = 'IT';
```

7. GROUP BY & HAVING

```sql
SELECT department, COUNT(*), SUM(salary), AVG(salary)
FROM employees
GROUP BY department
HAVING COUNT(*) > 5;
```

WHERE	HAVING
Individual rows filter karta hai	Groups filter karta hai
GROUP BY se pehle	GROUP BY ke baad
Normal conditions	Aggregate conditions

8. CASE, COALESCE, NULLIF

```sql
SELECT name, salary,
    CASE
        WHEN salary >= 80000 THEN 'High'
        WHEN salary >= 50000 THEN 'Medium'
        ELSE 'Low'
    END AS salary_level
FROM employees;

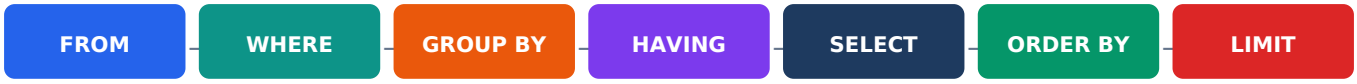
SELECT name, COALESCE(phone, 'Not Available') AS phone FROM customers;
SELECT sales / NULLIF(quantity, 0) FROM products;   -- avoid div by zero
```

9. String & Date Functions

```sql
UPPER(name)  LOWER(name)  LENGTH(name)  TRIM(name)
CONCAT(first_name, ' ', last_name)   -- or first_name || ' ' || last_name
SUBSTRING(name, 1, 3)  REPLACE(name, 'a', 'x')

CURRENT_DATE  CURRENT_TIMESTAMP  NOW()
EXTRACT(YEAR FROM created_at)
CURRENT_DATE + INTERVAL '7 days'
CURRENT_DATE - INTERVAL '30 days'
```

10. Logical Query Execution Order (Interview Gold)



FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT

```sql
SELECT department, AVG(salary) AS average_salary
FROM employees
WHERE salary > 30000
GROUP BY department
HAVING AVG(salary) > 55000
ORDER BY average_salary DESC
LIMIT 3;
```

11. Part 2 Practice Set

employees table par: Highest / lowest salary, Top 5 paid, Unique departments, IT+HR, Salary range, Dept-wise count / total / avg, Departments with >3 employees, Uppercase names, COALESCE phone, CASE salary levels, ORDER BY, Pagination.

3 PART 3 — JOINs, Relationships & Subqueries

1. Why JOINs?

Real-world data multiple tables mein hota hai. Customers + Orders ko combine karke customer name + order details chahiye → JOIN.

Relationships

Type	Example
One-to-One	Person → Passport
One-to-Many	Customer → Orders (most common)
Many-to-Many	Students ↔ Courses (via junction table)

2. JOIN Types Overview



INNER JOIN — Only Matching

```sql
SELECT c.name, o.order_id, o.amount
FROM customers AS c
INNER JOIN orders AS o ON c.customer_id = o.customer_id;
-- JOIN alone means INNER JOIN
```

LEFT JOIN — All Left + Matching Right

```sql
SELECT c.name, o.order_id
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id;
-- Customers without orders → order_id = NULL
```

RIGHT / FULL OUTER / CROSS / SELF

RIGHT JOIN — all right + matching left. Practical mein LEFT JOIN zyada use hota hai (tables swap karke).

FULL OUTER JOIN — both tables ki saari rows.

CROSS JOIN — every combination (A × B). Accidental use se huge result set ban sakta hai.

```sql
-- SELF JOIN (employee-manager)
SELECT e.name AS employee, m.name AS manager
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.employee_id;
```

3. Multiple JOINs + Aggregation

```sql
SELECT c.name AS customer, o.order_id, p.product_name, o.quantity
FROM customers c
JOIN orders o   ON c.customer_id = o.customer_id
JOIN products p ON o.product_id = p.product_id;
```

```sql
-- Customer-wise total spending
SELECT c.customer_id, c.name, SUM(o.amount) AS total_spending
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.name;
```

Customers with Zero Orders (Classic Interview)

```sql
SELECT c.customer_id, c.name
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

TIP

COUNT(o.order_id) use karo COUNT(*) ki jagah jab LEFT JOIN se zero-order customers count kar rahe ho — NULL-extended rows count nahi hongi.

ON vs WHERE with LEFT JOIN

⚠ IMPORTANT

Condition ON clause mein → left rows preserve rehti hain. WHERE mein lagane se LEFT JOIN effectively INNER jaisa behave kar sakta hai.

4. Subqueries

```sql
-- Employees earning above average
SELECT * FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);

-- Customers who placed at least one order
SELECT * FROM customers
WHERE customer_id IN (SELECT customer_id FROM orders);

-- EXISTS / NOT EXISTS (often safer than NOT IN with NULLs)
SELECT * FROM customers c
WHERE EXISTS (
    SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id
);
```

5. CTE (Common Table Expression)

```sql
WITH high_value_orders AS (
    SELECT * FROM orders WHERE amount > 50000
)
SELECT * FROM high_value_orders;

WITH customer_sales AS (
    SELECT customer_id, SUM(amount) AS total_spending
    FROM orders GROUP BY customer_id
)
SELECT * FROM customer_sales WHERE total_spending > 100000;
```

Multiple CTEs chain kiye ja sakte hain. Recursive CTE hierarchical data (org chart, categories) ke liye.

6. Part 3 Practice

Customer + orders display, All customers incl. no orders, No-order customers, Total spending per customer, Highest spenders, Order counts, >50k orders, Above-avg orders, EXISTS, 3-table JOIN.

4 PART 4 — Window Functions, CTEs, Views, Indexes & Transactions

1. Window Functions

Group of rows par calculation without reducing row count. GROUP BY rows reduce karta hai; Window har row ke saath result deta hai.

```sql
SELECT employee_name, department, salary,
       AVG(salary) OVER (PARTITION BY department) AS dept_avg
FROM employees;
```

Syntax: `function() OVER (PARTITION BY col ORDER BY col)`

ROW_NUMBER, RANK, DENSE_RANK

Function	Equal values	Next rank
ROW_NUMBER()	Separate numbers	Continues (1,2,3,4)
RANK()	Same rank	Gap (1,2,2,4)
DENSE_RANK()	Same rank	No gap (1,2,2,3)

```sql
SELECT employee_name, salary,
       RANK() OVER (ORDER BY salary DESC) AS rnk
FROM employees;
```

Top-N per Group Pattern

```sql
WITH ranked AS (
    SELECT *, ROW_NUMBER() OVER (
        PARTITION BY department ORDER BY salary DESC
    ) AS rn
    FROM employees
)
SELECT * FROM ranked WHERE rn <= 2;
```

LAG / LEAD, Running Total, Moving Average

```sql
SELECT sale_date, sales,
       LAG(sales) OVER (ORDER BY sale_date) AS prev,
       sales - LAG(sales) OVER (ORDER BY sale_date) AS diff,
       LEAD(sales) OVER (ORDER BY sale_date) AS next,
       SUM(sales) OVER (ORDER BY sale_date) AS running_total,
       AVG(sales) OVER (
           ORDER BY sale_date
           ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
       ) AS moving_avg_3
FROM daily_sales;
```

2. Views & Materialized Views

```sql
CREATE VIEW customer_orders AS
SELECT c.customer_id, c.name, o.order_id, o.amount
FROM customers c JOIN orders o ON c.customer_id = o.customer_id;

CREATE OR REPLACE VIEW ... ;
DROP VIEW customer_orders;

-- Materialized (physically stored result)
CREATE MATERIALIZED VIEW monthly_sales AS
SELECT DATE_TRUNC('month', order_date) AS month, SUM(amount)
FROM orders GROUP BY 1;
REFRESH MATERIALIZED VIEW monthly_sales;
```

3. Indexes

Fast searching ke liye data structure (book index jaisa).

```sql
CREATE INDEX idx_customers_email ON customers(email);
CREATE INDEX idx_orders_customer_id ON orders(customer_id);
CREATE INDEX idx_customer_orders ON orders(customer_id, order_date);  -- composite
CREATE UNIQUE INDEX idx_users_email ON users(email);
CREATE INDEX idx_active_users ON users(email) WHERE is_active = TRUE; -- partial
CREATE INDEX idx_lower_email ON users(LOWER(email));                  -- expression
```

4. Transactions & ACID




```sql
BEGIN;
UPDATE accounts SET balance = balance - 10000 WHERE account_id = 1;
UPDATE accounts SET balance = balance + 10000 WHERE account_id = 2;
COMMIT;          -- success
-- or ROLLBACK;  -- undo everything

-- SAVEPOINT for partial undo
SAVEPOINT transfer_step;
ROLLBACK TO SAVEPOINT transfer_step;
```

5. Classic Interview Patterns

SQL

```sql
-- Second highest salary
SELECT MAX(salary) FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);

-- or with window
SELECT * FROM (
    SELECT *, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
    FROM employees
) t WHERE rnk = 2;

-- Department-wise highest salary
SELECT * FROM (
    SELECT *, RANK() OVER (
        PARTITION BY department ORDER BY salary DESC
    ) AS rnk FROM employees
) t WHERE rnk = 1;
```

5 PART 5 — Functions, Triggers, JSONB, Arrays & Optimization

1. SQL & PL/pgSQL Functions

```SQL
CREATE FUNCTION add_numbers(a INTEGER, b INTEGER)
RETURNS INTEGER LANGUAGE SQL AS $$
    SELECT a + b;
$$;

CREATE FUNCTION calculate_bonus(salary NUMERIC)
RETURNS NUMERIC LANGUAGE plpgsql AS $$
BEGIN
    IF salary >= 100000 THEN
        RETURN salary * 0.20;
    ELSE
        RETURN salary * 0.10;
    END IF;
END;
$$;
```

2. Stored Procedures

```SQL
CREATE PROCEDURE update_salary(emp_id INTEGER, new_salary NUMERIC)
LANGUAGE SQL AS $$
    UPDATE employees SET salary = new_salary WHERE employee_id = emp_id;
$$;
CALL update_salary(101, 75000);
```

Function	Procedure
SELECT se call	CALL se call
Value return kar sakta hai	Primarily side-effects
Queries ke andar use	Standalone business operations

3. Triggers

Automatic action on INSERT / UPDATE / DELETE.

```SQL
CREATE FUNCTION log_employee_update()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    INSERT INTO employee_audit (employee_id, action_time)
    VALUES (NEW.employee_id, CURRENT_TIMESTAMP);
    RETURN NEW;
END;
$$;

CREATE TRIGGER employee_update_trigger
AFTER UPDATE ON employees
FOR EACH ROW EXECUTE FUNCTION log_employee_update();
```

NEW = new/current row, OLD = previous row. Useful for audit logs, salary history, etc.

4. JSONB

```sql
CREATE TABLE candidates (
    candidate_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name VARCHAR(100),
    profile JSONB
);

INSERT INTO candidates (name, profile) VALUES (
    'Gaurav', '{"skills": ["Python","SQL","ML"], "experience": 2}'
);

SELECT profile->'skills'   FROM candidates;   -- JSON
SELECT profile->>'skills'  FROM candidates;   -- text
SELECT * FROM candidates WHERE profile @> '{"experience": 3}';
SELECT * FROM candidates
WHERE (profile->>'experience')::INTEGER >= 2;

CREATE INDEX idx_candidate_profile ON candidates USING GIN(profile);
```

5. Arrays

```sql
CREATE TABLE students (
    student_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name VARCHAR(100),
    skills TEXT[]
);
INSERT INTO students (name, skills)
VALUES ('Gaurav', ARRAY['Python','SQL','Machine Learning']);

SELECT skills[1] FROM students;                -- 1-based
SELECT * FROM students WHERE 'Python' = ANY(skills);
SELECT name, UNNEST(skills) AS skill FROM students;
```

6. EXPLAIN & Query Optimization

```sql
EXPLAIN ANALYZE SELECT * FROM customers WHERE email = 'abc@gmail.com';
```

Scan Type	Meaning
Sequential Scan	Full table scan
Index Scan	Uses index
Index Only Scan	Mostly from index
Bitmap Index Scan	Bitmap-based lookup

Optimization Tips

- Avoid unnecessary SELECT * — only needed columns
- Appropriate indexes on WHERE / JOIN / ORDER BY columns
- Filter early (date ranges, status) on large tables
- Functions on indexed columns se index use nahi hota — expression index consider karo
- Always check EXPLAIN ANALYZE for real plans

7. Normalization (1NF / 2NF / 3NF)

Goal: Data duplication aur update anomalies reduce karna.

- 1NF — Atomic values, no repeating groups
- 2NF — 1NF + no partial dependency (composite PK cases)
- 3NF — 2NF + no transitive dependency of non-key attributes

E-commerce example: customers | products | orders | order_items (proper foreign keys)

8. PostgreSQL Power Features

SQL

```
-- UPSERT
INSERT INTO users (email, name) VALUES ('abc@gmail.com', 'Gaurav')
ON CONFLICT (email) DO UPDATE SET name = EXCLUDED.name;

-- RETURNING
INSERT INTO customers (name, email)
VALUES ('Gaurav', 'g@gmail.com')
RETURNING customer_id;

UPDATE customers SET name = 'Rahul' WHERE customer_id = 10 RETURNING *;
DELETE FROM customers WHERE customer_id = 10 RETURNING *;
```

9. Real-World E-Commerce Schema Snapshot

SQL

```
customers (customer_id PK, name, email UNIQUE, created_at)
products  (product_id PK, product_name, category, price, stock)
orders    (order_id PK, customer_id FK, order_date)
order_items (order_item_id PK, order_id FK, product_id FK, quantity, price)
```

Sample Interview Queries

SQL

```
-- Each customer's total spending
SELECT c.customer_id, c.name,
       SUM(oi.quantity * oi.price) AS total_spending
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
JOIN order_items oi ON o.order_id = oi.order_id
GROUP BY c.customer_id, c.name
ORDER BY total_spending DESC;

-- Products never ordered
SELECT p.product_id, p.product_name
FROM products p
LEFT JOIN order_items oi ON p.product_id = oi.product_id
WHERE oi.product_id IS NULL;

-- Top 2 products per category
WITH ranked AS (
    SELECT *, ROW_NUMBER() OVER (
        PARTITION BY category ORDER BY price DESC
    ) AS rn FROM products
)
SELECT * FROM ranked WHERE rn <= 2;
```

Complete Learning Roadmap

Level	Topics
1–4 Fundamentals	DB, tables, types, constraints, CRUD
5–7 Filtering & Agg	WHERE, GROUP BY, HAVING, aggregates
8–10 JOINs & Subq	INNER/LEFT/..., EXISTS, CTE
11–14 Advanced	Windows, Views, Indexes, Transactions
15–22 Expert	Functions, Triggers, JSONB, Arrays, Opt, Design